SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 4: Iteration and Loops — Automating Repetitive Engineering Tasks

---

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x and Google Colaboratory
Location: Morogoro, Tanzania

This lecture focuses on the power of repetition. In engineering, we often deal with large datasets (time-series rainfall, soil sensor arrays) or tasks that must be performed identically across thousands of iterations. Loops allow us to write a few lines of code that can process millions of data points.

Contents

1 Introduction: The Logic of Repetition **2**

2 The `for` Loop: Definite Iteration **2**
2.1 Syntax and Indentation . . . 2
2.2 The `range()` Function: Generating Sequences . . . 2

3 The `while` Loop: Indefinite Iteration **2**
3.1 The "Sentinel" Logic . . . 3
3.2 Safety Warning: The Infinite Loop . . . 3

4 Loop Control: `break`, `continue`, and `pass` **3**
4.1 The `break` Statement . . . 3
4.2 The `continue` Statement . . . 3

5 Accumulators and Counters: The Math of Loops **3**
5.1 Summation Logic ($\sum$) . . . 4

6 Advanced Topic: Nested Loops **4**

7 Case Study: 7-Day Cumulative Rainfall Analysis **4**

8 Case Study: Automated Grain Quality Control **5**

9 Debugging Loops: Common Mistakes **5**
9.1 1. The Off-By-One Error . . . 5
9.2 2. Forgetting to Update the Condition . . . 5
9.3 3. Initialization Errors . . . 5

10 Conclusion **5**

11 Glossary of Terms **7**

12 Extended Self-Assessment Exercises **8**
12.1 Exercise 1: Growing Degree Days (GDD) Calculator . . . 8
12.2 Exercise 2: Automated Pumping Limit . . . 8
12.3 Exercise 3: Sensor Grid Averaging . . . 8
12.4 Exercise 4: Livestock Inventory Scanner . . . 8

1 Introduction: The Logic of Repetition

In previous modules, we learned how to store data in variables and make single decisions using `if` statements. However, engineering problems are rarely solved by a single calculation. Agricultural systems often require us to perform the same task multiple times:

- **Data Aggregation:** Summing 365 days of temperature data to calculate "Growing Degree Days" (GDD).
- **Spatial Analysis:** Checking the soil moisture of 1,000 different nodes in a precision irrigation network.
- **Control Systems:** A greenhouse ventilator that checks the temperature every second, 24 hours a day.

Without loops, we would have to copy and paste our code thousands of times—a process that is inefficient and prone to human error. **Iteration** (or looping) allows a program to repeat a specific block of code as long as a condition is met or until a sequence is exhausted.

2 The `for` Loop: Definite Iteration

The `for` loop is used when you know in advance how many times you want to repeat a task. In engineering terms, this is often used for iterating over a fixed set of measurements or a specific range of implement settings.

2.1 Syntax and Indentation

Like the `if` statement, the `for` loop uses a colon (`:`) and indentation to define its scope.

```
# Iterating over a simple list of farm blocks
farm_blocks = ["Block A", "Block B", "Block C", "Block D"]

for block in farm_blocks:
    print(f"Inspecting irrigation lines in: {block}")
```

2.2 The `range()` Function: Generating Sequences

The `range()` function is indispensable in engineering simulations. It generates a sequence of numbers.

- `range(5)`: Generates 0, 1, 2, 3, 4.
- `range(1, 11)`: Generates 1 to 10.
- `range(0, 10, 2)`: Generates 0, 2, 4, 6, 8 (Starts at 0, ends before 10, increments by 2).

```
# Simulation: Increasing pump pressure in steps
for pressure in range(10, 51, 10):
    print(f"Testing pump at {pressure} PSI...")
```

3 The `while` Loop: Indefinite Iteration

The `while` loop repeats as long as a certain Boolean condition is `True`. This is used when we don't know exactly how many times we need to repeat, but we know when we should **stop**.

3.1 The "Sentinel" Logic

In monitoring systems, we might use a `while` loop to keep a system running until a critical failure or threshold is reached.

```
tank_level = 0.0 # Start empty
capacity = 100.0 # Max capacity

while tank_level < capacity:
    print(f"Filling... Current level: {tank_level}L")
    tank_level += 15.5 # Adding water in steps

print("Tank Full. Closing Valve.")
```

3.2 Safety Warning: The Infinite Loop

If the condition of a `while` loop never becomes `False`, the program will run forever. In real-world engineering, this can cause a computer to crash or, worse, cause hardware (like a motor) to overheat if there is no safety "break." *Example of a dangerous loop:* `while True: print("Pumping...")`

4 Loop Control: `break`, `continue`, and `pass`

Sometimes we need to interrupt the natural flow of a loop based on an internal condition.

4.1 The `break` Statement

This exits the loop immediately. Use this for emergency stops or when a search is complete.

```
for batch in range(1, 101):
    if batch == 45: # Found a contaminated batch
        print(f"CRITICAL: Contamination in batch {batch}. Halting line!")
        break
    print(f"Processing batch {batch}")
```

4.2 The `continue` Statement

This skips the rest of the current iteration and jumps back to the top of the loop for the next cycle. Use this to skip "bad data" points.

```
# Skip negative (invalid) rainfall readings
rainfall_data = [5.2, -1.0, 4.8, 0.0, 12.1]

for reading in rainfall_data:
    if reading < 0:
        continue # Skip the rest and go to next reading
    print(f"Valid reading: {reading}mm")
```

5 Accumulators and Counters: The Math of Loops

A common task in agricultural engineering is to calculate a total (sum) or a count of events. This requires the use of an **Accumulator Variable**.

5.1 Summation Logic ($\sum$)

Mathematically, calculating total rainfall over n days is:

$$Total = \sum_{i=1}^{n} R_i$$

In Python, we initialize a variable to zero **before** the loop and add to it **inside** the loop.

```
daily_rainfall = [12.5, 0.0, 5.2, 18.1, 0.0, 2.1, 1.5]
total_rainfall = 0 # Initialize

for rain in daily_rainfall:
    total_rainfall += rain # Add each day to the total

print(f"7-Day Cumulative Rainfall: {total_rainfall} mm")
```

6 Advanced Topic: Nested Loops

A nested loop is a loop inside another loop. This is essential for processing 2D structures, such as a field divided into a grid of plots (rows and columns).

```
# A field with 3 rows and 4 columns of sensors
for row in range(1, 4):
    for col in range(1, 5):
        print(f"Reading Sensor at Row {row}, Col {col}")
```

In this example, for every single iteration of the "outer" (row) loop, the "inner" (column) loop runs to completion. Total readings = $3 \times 4 = 12$.

7 Case Study: 7-Day Cumulative Rainfall Analysis

Engineering Context: Irrigation engineers use cumulative rainfall to determine if a crop's weekly water requirement has been met. If the 7-day total is below a threshold, the irrigation system must make up the deficit.

```
# --- Inputs ---
weekly_data_mm = [0.0, 5.5, 12.0, 0.0, 0.0, 2.5, 1.5]
crop_weekly_need_mm = 35.0
total_weekly_rain = 0.0

# --- Iterative Processing ---
print("--- Daily Rainfall Log ---")
day_count = 1
for rain in weekly_data_mm:
    print(f"Day {day_count}: {rain} mm")
    total_weekly_rain += rain
    day_count += 1

# --- Decision Support ---
print("\n--- Summary Analysis ---")
print(f"Total Weekly Rainfall: {total_weekly_rain} mm")

if total_weekly_rain >= crop_weekly_need_mm:
    print("STATUS: Rainfall sufficient. Irrigation NOT required.")
else:
    deficit = crop_weekly_need_mm - total_weekly_rain
    print(f"STATUS: Rainfall deficit of {deficit} mm detected.")
    print(f"ACTION: Schedule irrigation for {deficit} mm depth.")
```

8 Case Study: Automated Grain Quality Control

Engineering Context: A grain sorting machine uses a sensor to check the ”Whiteness” of rice grains. If the average whiteness of a sample falls below a threshold, the batch is downgraded.

```
# Simulated sensor readings for a batch of 10 samples
whiteness_readings = [88, 85, 92, 70, 89, 84, 91, 75, 88, 90]
sum_whiteness = 0
count = 0

for val in whiteness_readings:
    # If we find a critically damaged grain (below 50), stop and reject
    if val < 50:
        print("CRITICAL ERROR: Severe damage detected. Batch rejected.")
        break

    sum_whiteness += val
    count += 1

# Calculate Average
if count > 0:
    average_whiteness = sum_whiteness / count
    print(f"Batch Processing Complete. Average Whiteness: {average_whiteness}")

    if average_whiteness >= 85:
        print("GRADE: Premium Export Grade")
    else:
        print("GRADE: Standard Local Grade")
```

9 Debugging Loops: Common Mistakes

9.1 1. The Off-By-One Error

Engineers often forget that `range(10)` ends at 9, not 10. If you need 1 to 10, you must use `range(1, 11)`.

9.2 2. Forgetting to Update the Condition

In a `while` loop, if you forget to change the variable being checked, the loop becomes infinite.

```
# INCORRECT
x = 0
while x < 5:
    print("Counting...")
    # x += 1 is missing! This runs forever.
```

9.3 3. Initialization Errors

Forgetting to set `total = 0` before a loop will cause a `NameError` or, if the variable existed from a previous script, will result in an incorrect ”cumulative” sum.

10 Conclusion

Loops allow agricultural engineers to scale their logic. A program that calculates irrigation for one plot can easily be scaled to calculate for 10,000 plots using a loop. By mastering `for` and

`while` structures, you move from basic scripts to robust automation. In the next lecture, we will learn how to wrap these patterns into **Functions** to create modular, reusable engineering tools.

11 Glossary of Terms

- **Accumulator:** A variable used in a loop to accumulate a running total of values.
- **Break:** A keyword used to prematurely exit a loop based on a specific condition.
- **Continue:** A keyword used to skip the remainder of the current loop iteration and move to the next.
- **Definite Iteration:** A loop where the number of repetitions is known before the loop starts (usually a `for` loop).
- **Indefinite Iteration:** A loop where the number of repetitions depends on a condition being met during execution (usually a `while` loop).
- **Iteration:** A single pass through a block of code within a loop.
- **Nested Loop:** A loop structure placed inside another loop.
- **Sentinel Value:** A special value used to signal the end of a loop or data sequence.

12 Extended Self-Assessment Exercises

12.1 Exercise 1: Growing Degree Days (GDD) Calculator

GDD is a measure of heat accumulation used to predict plant development. Formula: $GDD = (T_{max} + T_{min})/2 - T_{base}$. **Task:** Given a list of daily maximum temperatures (`[30, 32, 28, 35, 33]`) and minimum temperatures (`[20, 22, 19, 24, 21]`), and a $T_{base} = 10$, write a script that calculates and prints the total GDD for these 5 days.

12.2 Exercise 2: Automated Pumping Limit

Write a `while` loop that simulates a water pump filling a 500L tank. The pump delivers 42L per minute. Every minute, the program should print the current volume. **Safety Constraint:** If the volume exceeds 480L, use a `break` statement to stop the pump early to prevent overflow.

12.3 Exercise 3: Sensor Grid Averaging

Consider a 3x3 grid of soil moisture sensors.

```
grid = [
    [25, 30, 22],
    [28, 24, 26],
    [20, 22, 21]
]
```

Task: Use a nested loop to calculate the average moisture across the entire field.

12.4 Exercise 4: Livestock Inventory Scanner

A list contains animal IDs: `[101, 102, "ERROR", 104, 105, 106, "ERROR", 108]`. **Task:** Use a `for` loop and a `continue` statement to print only the valid numeric IDs, skipping any entries that say "ERROR".

End of Lecture 4 Compendium